Machine Learning Based Analog Circuit Fault Diagnosis
PDF 20 — Chapter 27: day15_gridsearch.py Explained Line-by-Line
Parameter Grids | 5-Fold CV | best_params_, best_score_, best_estimator_

PART 1 — GridSearchCV Theory and Parameter Grids

Chapter 27 — Complete Python Source Code Explained Part 1 — day15_gridsearch.py (Based on your actual source code)

Before Starting

Your baseline SVM worked. But there was one problem. It used the default settings chosen by Scikit-Learn. Question. How does Scikit-Learn know the best settings for your dataset? It doesn't. The defaults are general-purpose values, not values optimized for your analog circuit dataset. That is why

Day 15

exists. Goal of This Script Professor asks What is the purpose of day15_gridsearch.py?

Perfect Answer

This script systematically searches for the optimal SVM hyperparameters using GridSearchCV and 5-fold cross-validation. It compares many parameter combinations on the training data, selects the best-performing model, and finally evaluates that optimized model on the untouched test set. What Problem Does GridSearchCV Solve? Imagine buying shoes. Question. Would you buy the first size you see? No. You try different sizes until one fits best. GridSearchCV does exactly the same thing. Instead of shoe sizes, it tries different hyperparameters. What Are Hyperparameters? Question. What is a hyperparameter? A hyperparameter is a setting chosen before training begins. Examples C gamma kernel degree The SVM does not learn these values. We choose them. Model Parameters vs Hyperparameters Very important.

Learned during training	Chosen before training
Decision boundary	C
Support vectors	gamma
Weights	kernel

PROFESSOR ASKS

What is the difference between a parameter and a hyperparameter?

EXCELLENT ANSWER

Model parameters are learned automatically during training, whereas hyperparameters are chosen before training begins and control how the learning algorithm behaves. Reading the Dataset The beginning of this script looks almost identical to Day 14. It Reads the CSV. Encodes labels. Splits data. Applies StandardScaler. Nothing changes here. The new part starts with GridSearchCV.

Search Grids

Your code creates three parameter grids. Linear Kernel "C": [0.001, 0.01, 0.1, 1, 10, 100, 1000] Question. Why only C? Because Linear SVM has only one important hyperparameter. No gamma. No degree. RBF Kernel Your code searches C × gamma Question. Why both? Because these two work together. Changing only C doesn't tell the full story. Some values of C work well only with certain values of gamma.

Polynomial Kernel

Your code searches C gamma degree coef0 This is the largest search space. Polynomial SVM has more hyperparameters than Linear or RBF. Why Log-Spaced Values? Question. Why 0.001 0.01 0.1 1 10 100 1000 instead of 1 2 3 4 5 Because SVM performance changes approximately on a logarithmic scale. Moving from 1 to 10 can make a huge difference. Moving from 1 to 2 often changes very little. Log spacing covers a much larger range efficiently.

PROFESSOR ASKS

Why are hyperparameters searched on a logarithmic scale?

EXCELLENT ANSWER

SVM performance often changes significantly across orders of magnitude rather than small linear increments. Logarithmic spacing efficiently explores both very small and very large values. How Many Models Are Trained? Let's calculate. Linear 7 values × **5 folds** — 35 fits. RBF **5 × 6** — 30 combinations. × **5 folds** — 150 fits. Polynomial **4 × 3 × 2 × 1** — 24 combinations. × **5 folds** — 120 fits. Total hundreds of SVM models are trained automatically. This is why GridSearchCV takes much longer than ordinary training. What Is Cross Validation? This is the heart of GridSearchCV. Your code uses cv = 5 Question. What does 5-fold mean? Suppose the training set contains 800 samples. Split it into 5 equal parts.

Fold 1

Fold 2

Fold 3

Fold 4

Fold 5

Training Process

Round 1

Train — 2. 3 4 5 **Test** — 1.

Round 2

Train — 1. 3 4 5 **Test** — 2. Repeat until every fold has been used once as validation. Question. Why? Because one train-test split may be lucky. Another may be unlucky. Cross-validation reduces this randomness.

PROFESSOR ASKS

Why use Cross Validation instead of one validation split?

EXCELLENT ANSWER

Cross-validation evaluates the model on multiple validation splits instead of only one. This provides a more reliable estimate of generalization performance and reduces dependence on a single random split. GridSearchCV Now your code creates GridSearchCV(.) Question. What happens? GridSearchCV automatically does this.

Choose Parameters

Train — Validate.

Record Score

Repeat

Choose Best

You do not write the loop. GridSearchCV writes it for you.

Very Important Point

Notice what your code does not do. It never touches the test set during GridSearchCV. Only the training data is used for searching. The test set is kept aside until the best model has already been selected.

PROFESSOR ASKS

Why is the test set never used during GridSearchCV?

EXCELLENT ANSWER

The test set must remain completely unseen while selecting hyperparameters. Otherwise, the model would indirectly optimize for the test data, leading to overly optimistic performance estimates.

Why This Script Is Important

Day 14

asked "How well does the default SVM work?"

Day 15

asks "Can we do better by choosing smarter hyperparameters?" That is a completely different question.

Complete Pipeline

Parameter Grid

5-Fold Cross Validation

Evaluate Every Combination

Choose Best Parameters

Train Best Model

Test Once

This is exactly what GridSearchCV automates.

COMMON MISTAKES

MISTAKE: [X] Tuning using the test set. Wrong.

MISTAKE: [X] Searching only one parameter. Wrong. Some hyperparameters interact.

MISTAKE: [X] Using very small search ranges. Wrong. Explore multiple orders of magnitude.

MISTAKE: [X] Thinking GridSearchCV changes the algorithm. Wrong. It changes only the hyperparameters.

REVISION NOTES

Remember the tuning workflow. **Baseline Model** — Parameter Grid. **Cross Validation** — Compare Scores. **Best Hyperparameters** — Final Model. End of Chapter 27 (Part 1) Congratulations. You now understand the purpose and theory behind GridSearchCV. You can explain: What hyperparameters are. Why GridSearchCV is needed. Why logarithmic search values are used. How 5-fold cross-validation works. Why the test set remains untouched. How GridSearchCV automatically evaluates hundreds of SVM models. Author's Note The next continuation will cover Chapter 27 (Part 2): Every GridSearchCV call in your code. Best parameter selection. best_estimator_ best_params_ best_score_ Comparing tuned Linear, RBF, and Polynomial kernels. Why RBF became your final model. How much each kernel improved over the baseline. The real engineering lesson from Day 15.

PART 2 — Understanding GridSearchCV Results

Chapter 27 — Complete Python Source Code Explained Part 2 — day15_gridsearch.py (Understanding GridSearchCV Results) (Based on your actual source code)

Before Starting

In Part 1,

we learned how GridSearchCV works. Now, we'll study what actually happens inside your code after GridSearchCV finishes searching. This is where your project finds its best SVM.

The Search Begins

Your code creates

```
gs_linear = GridSearchCV(...)
```

Then gs_linear.fit(X_train, y_train) Question. What happens after fit()? GridSearchCV starts training hundreds of models. Example

C = 0.001

Train — CV Score. Next

C = 0.01

Train — CV Score. Next

C = 0.1

Train — CV Score. It repeats until every parameter combination has been tested. How Does It Choose the Winner? Suppose

Linear Kernel

produces

C = 0.001

69%

C = 0.01

73%

C = 0.1

76%

C = 1

78%

C = 10

81%

C = 100

79% Question. Which one wins? Obviously

C = 10

because it produced the highest cross-validation accuracy. best_params_ Your code uses gs_linear.best_params_ Question. What is it? It returns the hyperparameters that produced the highest cross-validation score. Example { "C":100 } This means GridSearchCV has decided that

C = 100

is better than every other tested value.

PROFESSOR ASKS

What does best_params_ return?

EXCELLENT ANSWER

best_params_ returns the combination of hyperparameters that achieved the highest mean cross-validation score during the GridSearchCV search. best_score_ Your code also uses gs_linear.best_score_ Question. What is this value? It is the average cross-validation accuracy obtained using the winning hyperparameters. Example

Fold 1
82%

Fold 2
81%

Fold 3
83%

Fold 4
82%

Fold 5
84% **Average** — 82.4%. That becomes best_score_. Question. Notice something important. This is NOT the test accuracy. It is the cross-validation accuracy measured only on training data.

PROFESSOR ASKS

Is best_score_ the test accuracy?

EXCELLENT ANSWER

No. best_score_ is the average cross-validation accuracy obtained on the training data during GridSearchCV. Test accuracy is calculated later using the unseen test set. best_estimator_ Now comes the most useful object. Your code uses gs_linear.best_estimator_ Question. What is best_estimator_? It is the actual trained SVM model using the best hyperparameters. Instead of creating another SVM, GridSearchCV already gives you the best one. Example Conceptually GridSearchCV

Best Parameters

Best Trained Model
best_estimator_ Predicting With the Best Model Your code does y_pred = gs_linear.best_estimator_.predict(X_test) Question. Why? Because we don't want an average model. We want the best model found during the search. Now the test set is finally used.

Test Accuracy
Your code calculates accuracy_score(.) again. Exactly like Day 14. The difference is that this accuracy belongs to the optimized model, not the default model. Comparing Baseline vs Tuned Suppose Linear produced **Baseline** — 76%. **Tuned** — 79%. **Improvement** — 3%. That means GridSearchCV found better hyperparameters than the defaults. Your code prints something similar to

Baseline Accuracy
76%

Best Accuracy
79% **Improvement** — +3%. Why Compare Kernels? Your code runs GridSearchCV for three kernels. Linear RBF Polynomial Each gets its own optimized hyperparameters. Then you compare their final accuracies. Question. Why? Because we don't know which kernel fits the data best. The data decides.

Not us. Why RBF Won In your project, the RBF kernel showed the largest improvement after tuning, making it the strongest overall model. Question. Why? Because analog circuit fault boundaries are not perfectly linear. The RBF kernel can model complex, nonlinear decision boundaries, whereas the Linear kernel cannot.

PROFESSOR ASKS

Why was RBF selected as the final model?

EXCELLENT ANSWER

The tuned RBF kernel achieved the best overall classification performance because it can model nonlinear relationships between the extracted signal features, which better represent the behavior of analog circuit faults. Why Polynomial Didn't Win Polynomial is also nonlinear. Question. Then why didn't it win? Because higher complexity does not always mean better accuracy. Polynomial kernels can become too rigid or too sensitive, depending on the data. For your dataset, RBF matched the problem better.

Engineering Lesson

Notice what happened. You didn't change the dataset. You didn't add more features. You didn't change the algorithm. You simply chose better hyperparameters. That alone improved performance. This is the power of hyperparameter tuning.

Complete Pipeline

Parameter Grid

GridSearchCV

Cross Validation

Best Parameters

Best Estimator

Predict Test Set

Final Accuracy

This is exactly what

Day 15

does.

COMMON MISTAKES

MISTAKE: [X] Thinking best_score_ is test accuracy. Wrong. It is cross-validation accuracy.

MISTAKE: [X] Thinking best_estimator_ creates a new model. Wrong. It returns the already-trained best model.

MISTAKE: [X] Choosing a kernel before testing. Wrong. Always compare multiple kernels.

MISTAKE: [X] Assuming more complex kernels are always better. Wrong. The dataset decides.

REVISION NOTES

Remember these three objects. **best_params_** — Best Hyperparameters. **best_score_** — Best CV Accuracy. **best_estimator_** — Best Trained Model. These are the three most important outputs of GridSearchCV.

MEMORY TRICK

Remember this sentence. "Search → Score → Select." GridSearchCV always follows these three steps.

KNOWLEDGE CHECK

Level 1

- What does best_params_ return?
- What is best_score_?
- What is best_estimator_?
- Why compare three kernels?
- Why did RBF perform better?

Level 2

- Explain the difference between CV accuracy and test accuracy.
- Why is best_estimator_ used for prediction?
- Why are nonlinear kernels useful for analog circuit faults?
- Why does hyperparameter tuning improve performance?
- Describe the GridSearchCV workflow.

Professor Level

- Explain the outputs of GridSearchCV.
- Why should kernel selection be based on experimental results rather than assumptions?
- Why did the RBF kernel outperform the Linear kernel?
- What engineering insight did hyperparameter tuning provide in your project?
- Explain the complete tuning process from parameter grids to final prediction.
- End of Chapter 27 (Part 2)
- Congratulations.
- You now understand:
- How GridSearchCV evaluates parameter combinations.
- The meanings of best_params_, best_score_, and best_estimator_.
- How the best SVM is selected.
- Why RBF became the final classifier.
- How hyperparameter tuning improved your project without changing the dataset or feature engineering.
- Author's Note
- The next continuation begins Chapter 28 — day16_confusion_analysis.py.
- This is one of the most important chapters in the entire Project Bible.
- You'll learn:
- Reading and interpreting the confusion matrix.
- Finding the weakest classes using F1 scores.
- Discovering the RC_Open problem.
- Ranking the most confused class pairs.
- Feature comparison between Normal and Drift.
- How error analysis led to DC Level and later the Topology-Aware Rule.
- This chapter tells the detective story behind your biggest engineering improvement.